武漢大學
WUHAN UNIVERSITY

《分布式软件原理与技术》
第六讲 分布式共识

计算机学院　贾向阳

武漢大學
WUHAN UNIVERSITY

1、 什么是共识？

为什么需要多副本？

单机服务的问题

- 使用一个机器搭建的数据库/缓存服务，最大的问题是什么？
- **可用性问题**：如果这台服务器宕机了，服务就不可用了。
- **可靠性问题**：如果硬盘坏了，数据就丢失了。

多副本方案

- **可用性：**部署多个副本，一个宕机，其他副本仍可服务 → 需要负载均衡和故障转移。
- **可靠性：**数据复制多份，一份损坏，其他副本仍存在 → 需要复制协议（主从、集群）
- 新问题：多副本带来了**一致性挑战 —— 如何保证所有副本数据相同？**

多副本复制方法

（1）主从复制

- 读写分离架构中， 从节点通过异步或同步方式复制主节点的数据变更
- 主要问题：
  - 写可用性低：主节点宕机后，系统无法写入，需要人工或自动故障转移。
  - 数据不一致：异步复制下，客户端可能读到过时数据。

（2）多主复制

- 多个节点都可以处理写请求，彼此之间相互复制数据变更。
- 常见于分布式数据库（如 Cassandra、MySQL Group Replication）
- 主要问题：
  - 写冲突：两个客户端同时在不同主节点修改同一数据，会产生冲突。
  - 一致性弱：不同节点可能暂时读到不同版本的数据（最终一致性）。

多副本复制方法

（3）状态机复制

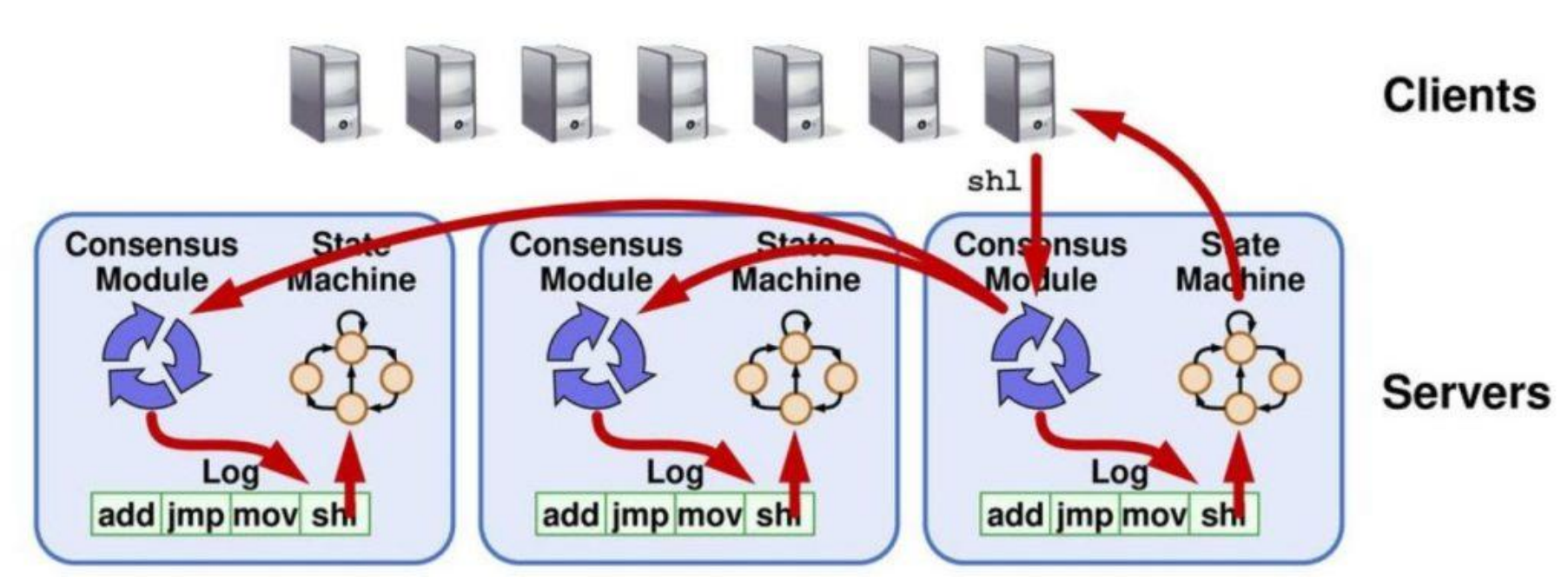
- 状态机复制是一种构建容错分布式系统的技术。
- 核心思想：在多个独立的节点上运行相同的状态机副本，通过一个共识协议确保所有状态机副本以相同顺序执行相同命令，从而保持节点状态一致
- 作用：在分区或服务器故障时，内部并不完全一致，但整体对外表现出强一致性
  - 容错：如果某个节点宕机，其他节点仍然可以继续服务，客户端不会感知到故障。
  - 强一致性：所有正常节点整体对外呈现完全相同的状态，就像只有一个状态机一样。
  - 可用性：只要多数节点存活，系统就能正常处理请求。

复制方法	复制的内容	顺序保证	一致性
主从复制	数据变更	无（可能乱序）	最终一致
多主复制	数据变更	冲突时需协调	最终一致
状态机复制	指令序列（日志）	全局顺序	线性一致（强）

状态机复制的难点

难点：状态机复制要求所有节点以相同的顺序执行相同的命令。在分布式系统中，节点分散、网络延迟不同、节点可能故障，如何**让所有节点对‘下一个要执行的命令是什么’达成一致**？

共识
Consensus



共识

共识问题

- 在一个分布式系统中，多个节点需要对某个值（或者某个命令、某个决策）达成一致。
- 在状态机复制场景中，所有节点需要对日志条目的序列达成一致。

实现共识的难点

- 理论上，如果分布式系统中各节点都能以十分“理想”的性能稳定运行，节点之间通信瞬时送达，则实现共识过程并不十分困难，简单地通过广播进行投票和应答即可。
- 可惜的是，现实中这样的“理想”系统并不存在。通信网络会发生中断、节点会发生故障，甚至存在被入侵的节点故意伪造消息，破坏正常的共识过程。

故障模型

- **宕机故障（Crash Fault）**
  - 节点可能停止工作，但不会发送错误信息（即“静默失败”）。
  - 节点可以重启，并可能从持久化存储中恢复之前的状态。
  - 这是最常见且最容易处理的故障模型，大部分共识算法就针对此。
- **拜占庭故障（Byzantine Fault）**
  - 节点可以任意行为：发送错误消息、撒谎、合谋攻击。
  - 需要拜占庭容错（BFT）算法，如 PBFT、HotStuff。
  - 现实中常见于区块链、军事、航天等需要防范恶意节点的场景。

共识算法

共识算法

- 在分布式系统中，共识算法使得多个节点能够就某个值（或决策）达成一致，即使在存在节点故障、网络延迟、消息丢失等不可靠因素的情况下。
- 共识算法是分布式系统实现**“强一致性”和“高可用性”的基石**，它让一群不可靠的节点能够像一个可靠的整体一样工作。
- 能够暂时容忍分布式系统内部存在不同状态，在外部看来始终表现出整体一致的结果。

共识算法分类

- Crash Fault Tolerance (CFT) ：如 Paxos、Raft及其变种等
- Byzantine Fault Tolerance（BFT）：如PBFT、Algorand等

本课程专注于CFT共识算法，因为这是大多数工业级分布式系统（如 ZooKeeper、etcd、Consul）的处理范畴

共识≠完美：FLP 不可能原理

- **历史背景**
  - 1985 年，Fischer、Lynch、Paterson 发表论文《Impossibility of Distributed Consensus with One Faulty Process》
- **“FLP 不可能原理”**
  - **定义**：在异步系统中，即使只有一个进程可能发生崩溃故障，也不存在一个确定性的共识算法能保证在有限时间内达成一致。
  - **理解**：任何一方都无法区分“对方崩溃”与“网络极慢”。因此它不敢擅自决定，结果就是：双方可能永远在“等待”中，无法终止。
  - **意义**：任何共识算法都存在一定概率无法实现一致。
  - 工程上，我们可以假设网络最终会稳定、消息延迟有上界，并引入随机性，打破“确定性”的限制。

2、 Paxos共识算法

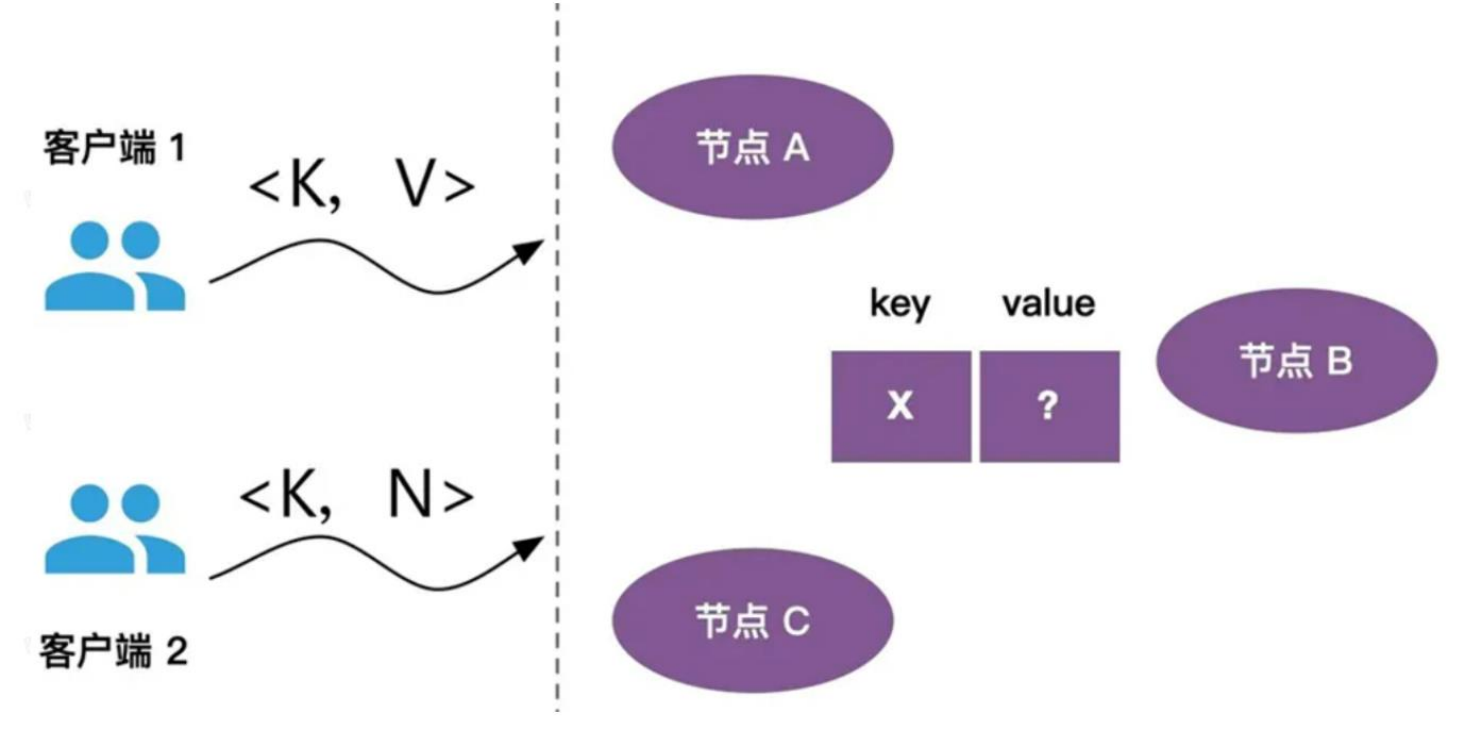
Paxos —— 分布式共识的“祖师爷”

- 诞生背景
  - 1990年由Leslie Lamport提出，但直到1998年才发表
- 核心价值
  - 第一个被数学证明正确的共识算法，奠定了所有后续协议的基础
  - ZooKeeper的Zab协议、Raft协议都受其启发
- 难点
  - 以“难以理解”著称，Lamport本人曾说：“几乎所有人都觉得难以理解”

资料：https://seven97.top/microservices/protocol/paxos-detail.html

Paxos解决什么问题？

- 例：客户端 1 和客户端 2 都试图更新x的值。集群如何达成共识，实现各节点上x的值一致呢？



“古希腊议会”

- 类比：
  - 一个古希腊城邦，公民大会需要通过一项法律。但是议员们分散在各个岛屿，通信依赖信使。信使可能延迟、丢失（网络分区），甚至有些议员可能因病缺席（节点故障）。
  - 议会需要保证：最终只能通过一项法律，并且所有议员都知道通过的是什么。

议员 = 分布式系统中的节点

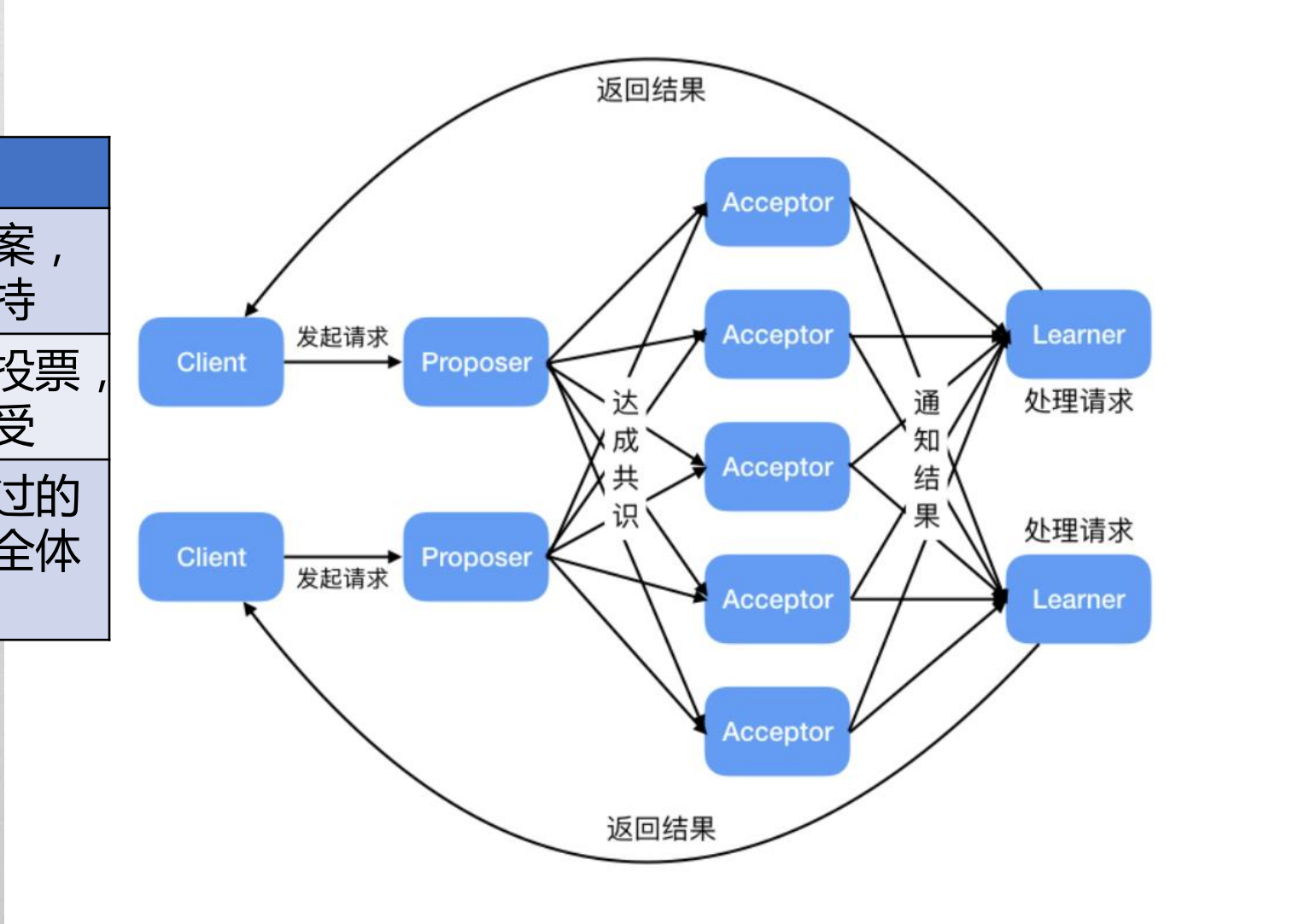
提案 = 节点中要写入的值

信使 = 网络消息

通过法律 = 达成共识（某个值被选中）

Paxos 的三个角色

Paxos 角色	比喻	职责
Proposer	提案人（如执政官）	提出法律草案，争取议员支持
Acceptor	议员	对提案进行投票，决定是否接受
Learner	公告员/书记官	记录最终通过的法律，告知全体公民



Paxos 的两阶段提交

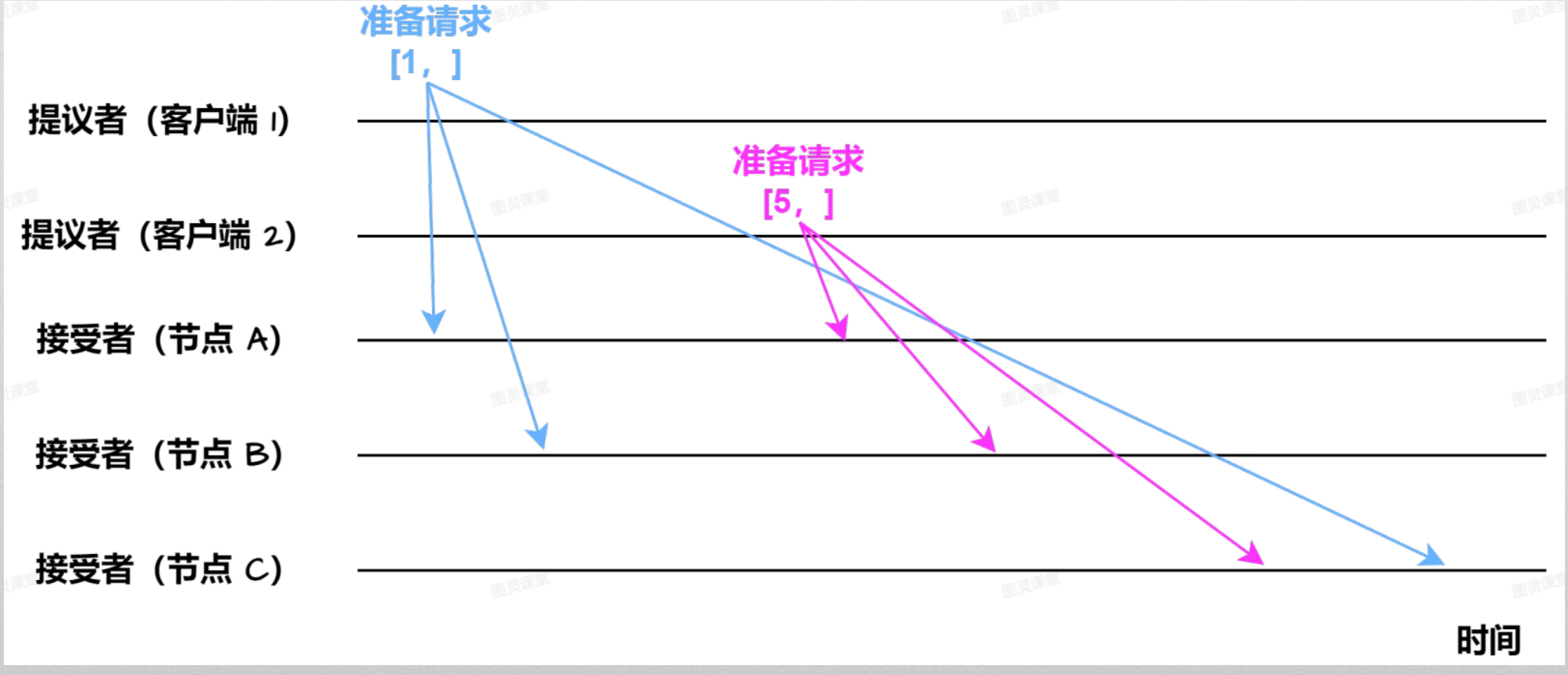
第一阶段：准备与承诺（Prepare / Promise）

- **提案人准备提案**：提案人想提出一项新法律，他先给每名议员派一名信使，带去一张羊皮纸，上面写着："我准备提出**编号为 n** 的提案，**请你们承诺**：不再接受任何编号小于 n 的提案，并把你们之前已经通过的最高编号提案告诉我。"
- **议员的承诺**：每个议员收到信使后，检查羊皮纸上的编号 n。如果 n 大于自己之前承诺过的最大编号，他就承诺：以后绝不接受编号小于 n 的任何提案。同时，他会把自己**已经投票通过的、编号最大的提案内容**（如果有的话）写在回信里，让信使带回给提案人。如果 n 小于等于已承诺过的编号，他就拒绝这个准备请求。
- **提案人收集响应**：提案人等待多数议员的回应。如果收到了**超过半数议员**的承诺（包括他们返回的已通过提案），第一阶段结束。

- **承诺接受新提案是为了保障算法的活性，避免无限等待旧提案的确认消息。**
- **返回已通过提案**，是让新提案人知道之前可能已经有法律被通过了，他要遵守。

Paxos算法的过程模拟

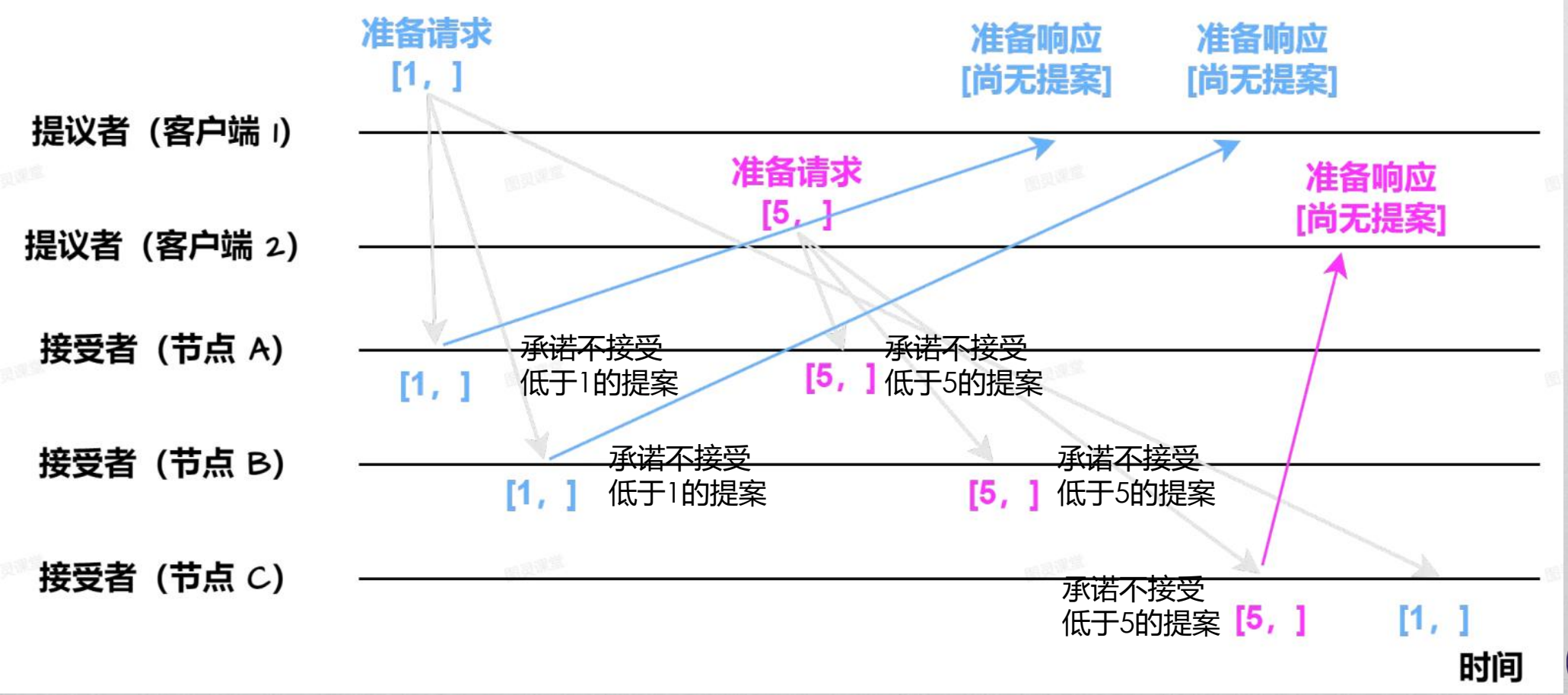
- 两个提议者先后发出“准备请求”，带上请求的编号



武漢大學
WUHAN UNIVERSITY

Paxos算法的过程模拟

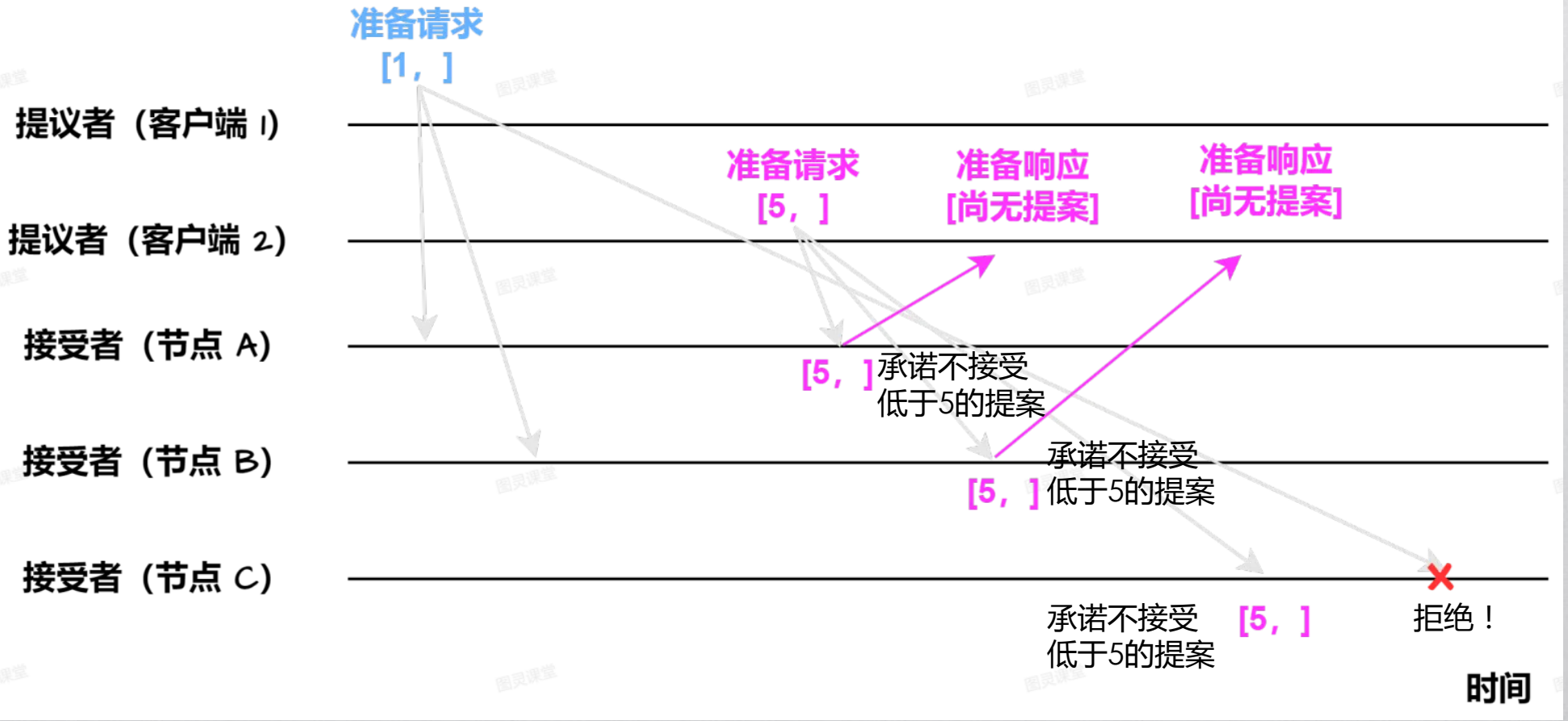
➢ 接受者返回承诺以及历史提案（当前没有提案）给提议者



武漢大學
WUHAN UNIVERSITY

Paxos算法的过程模拟

- 接受者C接受到提议者1的请求较晚，由于编号小于5，所以拒绝了提议者1。



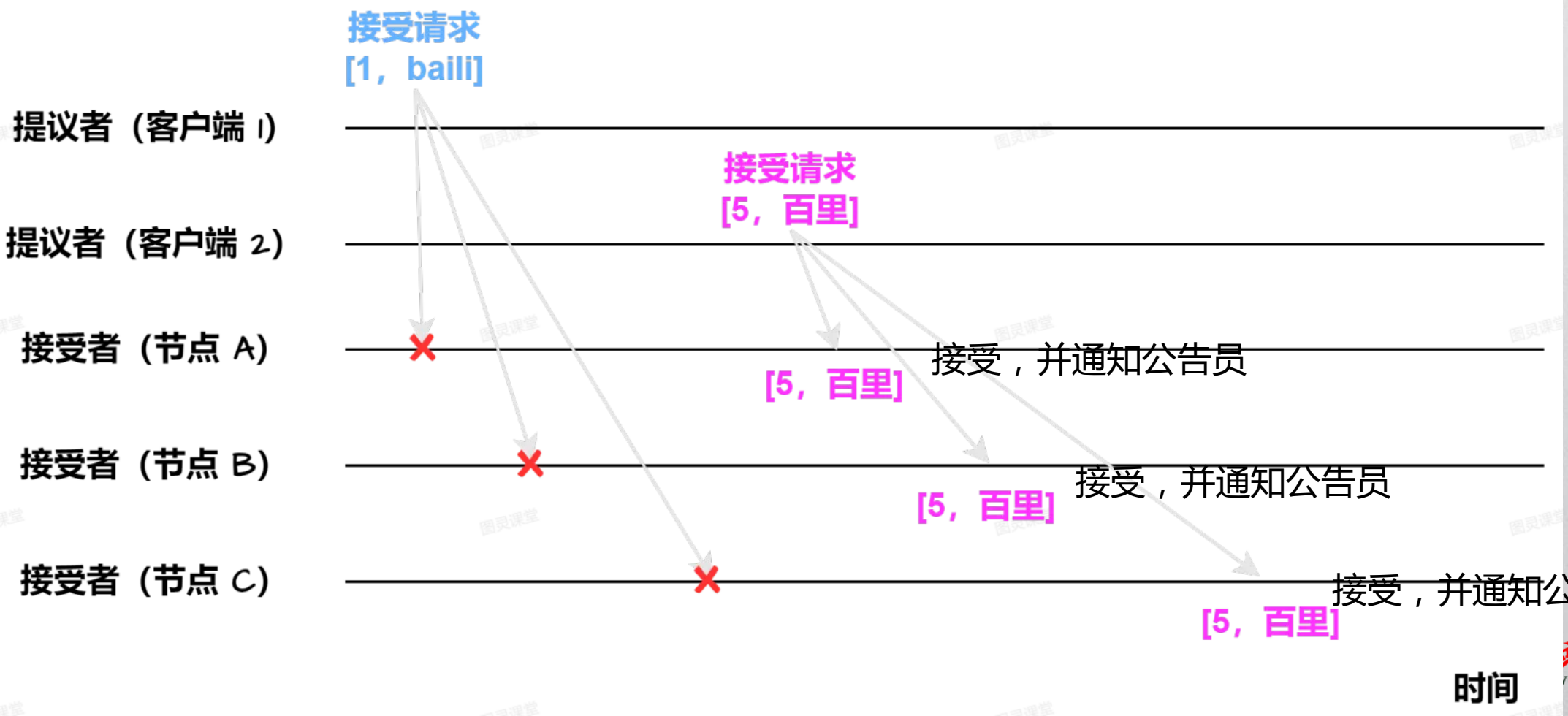
武漢大學 WUHAN UNIVERSITY

Paxos 的两阶段提交

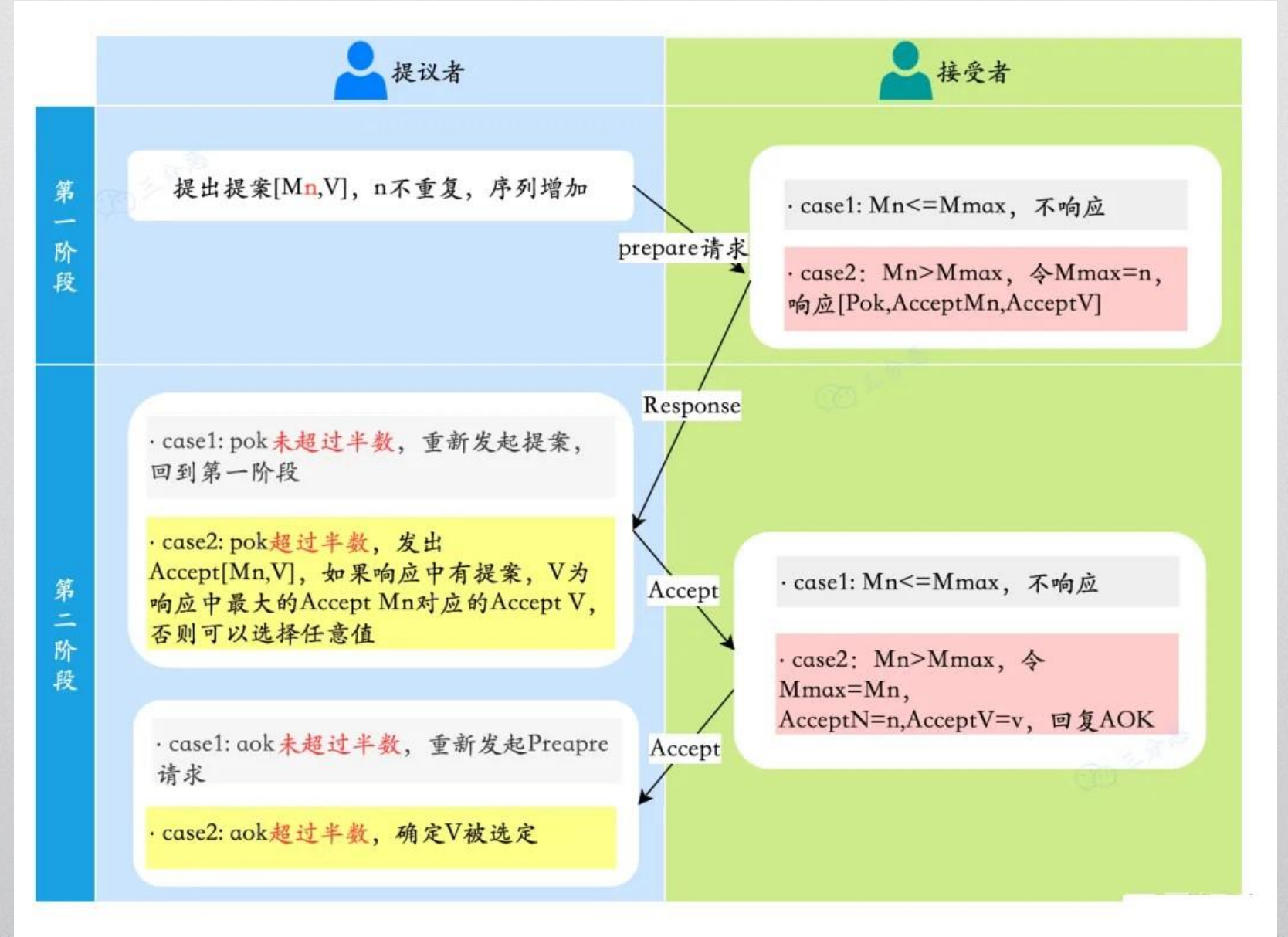
- **第二阶段：正式表决（Accept / Accepted）**
  - **提案人确定提案内容：**提案人查看所有返回的承诺中，如果没有任何一个议员返回过已通过提案，提案人可以自由选择自己希望的法律内容 v。如果有议员返回了已通过提案，那么提案人选择其中**编号最大的提案内容**作为自己的提案值 v。这样做是为了继承已通过的法律。
  - **提案人发起表决**：提案人再次派信使，向同样那批多数议员发送正式的表决请求：“请接受编号为 n 的提案，内容为 v。”
  - **议员正式表决：**议员收到表决请求后，检查编号 n 是否仍然不小于自己最后承诺的编号（可能在此期间又收到其他提案人更大编号的准备请求）。如果满足条件，议员就**接受该提案，记录下 (n, v)**，并派信使通知所有公告员（Learner）：“编号 n 的提案已被我接受。”如果不满足条件（例如已经承诺过更大的编号），则拒绝。
  - **公告员宣布法律：**当公告员（Learner）收到来自**多数议员的“已接受”**通知时，他就能确定提案 n 被正式通过了，于是向全体公民公布：“法律 v 正式生效！”

Paxos算法的过程模拟

- 两个提议者1的请求发送给三个接受者， 三个都接受了后者的值



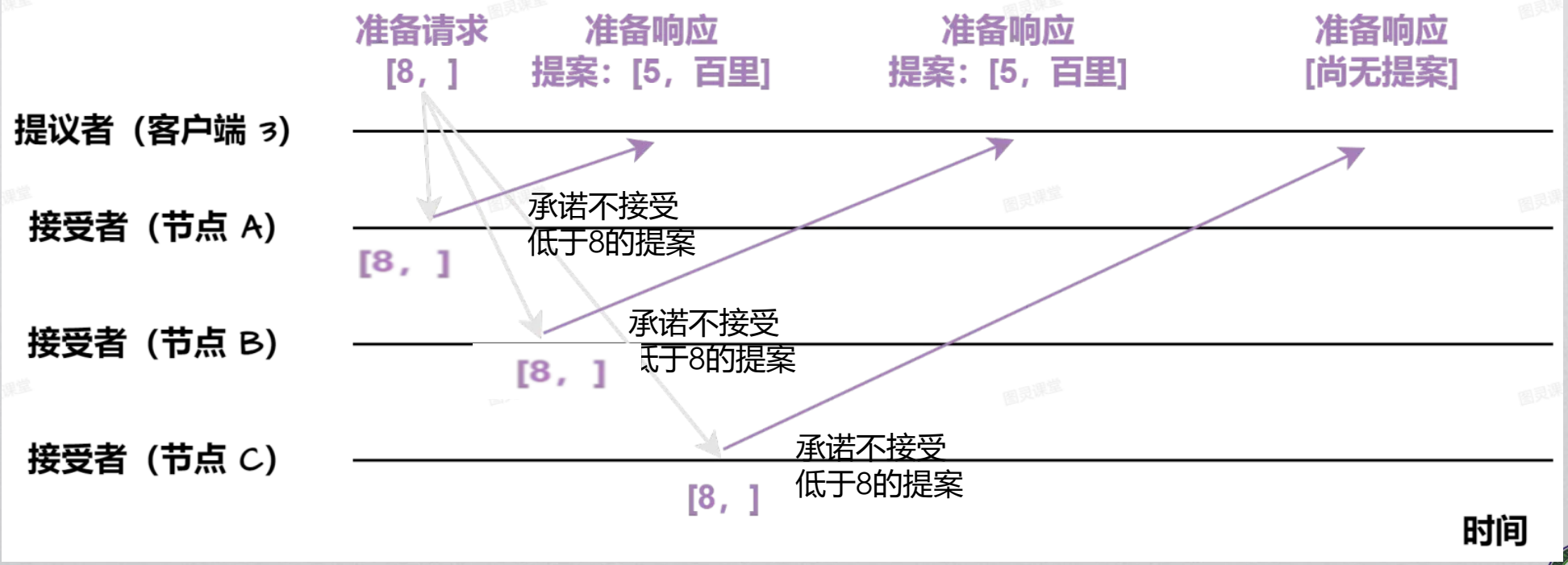
Paxos算法整体流程



武漢大學
WUHAN UNIVERSITY

Paxos算法的过程模拟

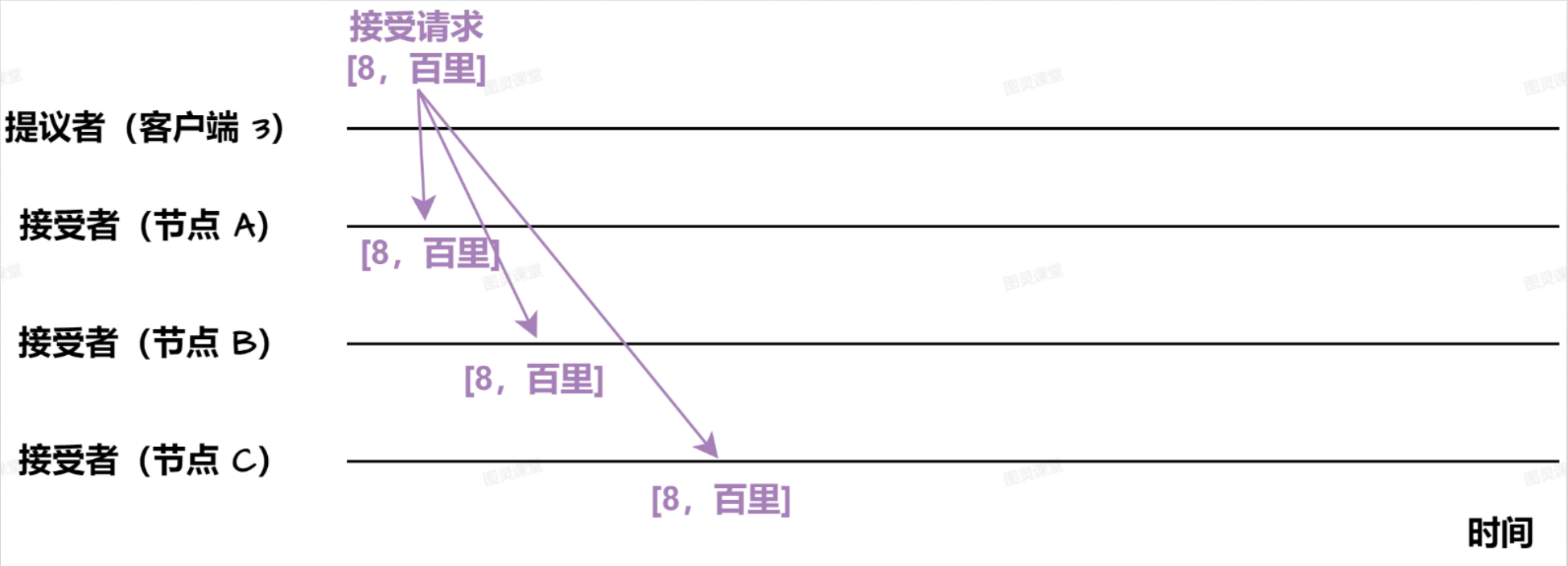
- 假设节点 A, B 已接受 [5, "百里"]（共识已形成），但节点C还没有，新的提议者客户端 3发出新提案为 [8，""]
- 接受者承诺将已取得的共识 [5, "百里"] 返回给提议者



武漢大學
WUHAN UNIVERSITY

Paxos算法的过程模拟

- 提议者客户端 3将已有历史共识“百里”，再发给所有接受者



武漢大學 WUHAN UNIVERSITY

关键规则

1. 多数派原则

- 无论是准备阶段还是表决阶段，提案人必须获得超过半数议员的响应才能继续。
- 多数派之间必然有交集，这就保证了不可能有两个不同的提案同时获得多数支持。

2. 承诺传递

- 议员一旦承诺“不接受小于 n 的提案”，就保证了任何后来的提案如果编号更大，必须包含之前已通过的值（通过第一阶段返回已通过提案实现）。
- 因此，一旦某个提案被通过，后续所有提案都只能沿用同一个值，保证了最终一致性。

关键规则

3. 编号机制

- 提案编号在 Paxos 中通常是全局递增的（如 (round, server_id) 组合）。
- 在比喻中，提案人可以选择任何编号，但必须保证每次准备请求的编号都比之前大。这避免了“乱序”导致的混乱。

4. 活锁问题

- 如果两个提案人交替提出编号不断增大的准备请求，议员就会不断承诺更大的编号，而永远无法进入正式表决阶段。
- 现实中，这需要引入随机退避或选举唯一的提案人（Leader）来避免。

3、 Raft共识算法

Raft算法

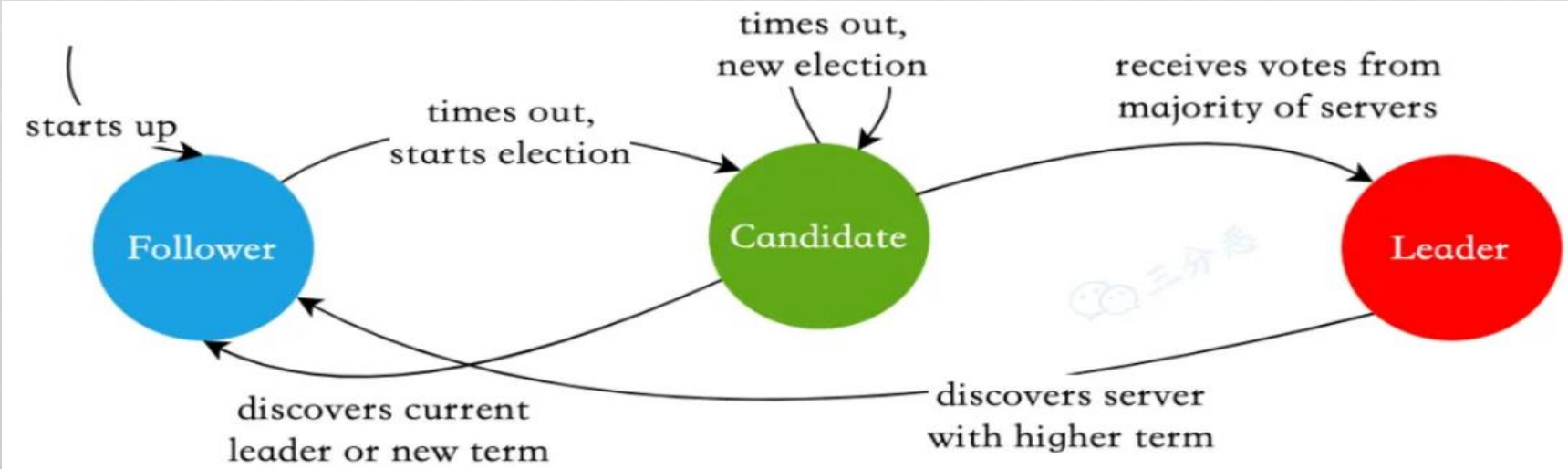
- **Paxos 的缺点**
  - 难以理解：Lamport 的论文抽象、角色不清、缺乏直观解释
  - 难以实现：缺少工程指导，成员变更、日志管理需自行设计
- **Raft算法**
  - 斯坦福大学的 Diego Ongaro 和 John Ousterhout 在 2013 年提出了 Raft
  - 不同于Paxos算法直接从分布式一致性问题出发推导出来，Raft算法则是从多副本状态机的角度提出，用于管理多副本状态机的日志复制
  - 目标就是——让共识算法能被大多数人理解和易于实现
    - 与 Paxos 同等正确性（Safety + Liveness）
    - 将共识问题分解为多个子问题，为工程实现提供清晰指导
    - 支持成员变更、日志压缩等实际需求

Raft算法的主要思想

- **主要思想**
  - 强领导者（Strong Leader）：系统中只有一个活跃 Leader，由它处理所有客户端请求
  - 领导任期（Term）：以任期作为逻辑时钟，新任期的领导可“压倒”旧任期的领导
  - 日志复制：Leader 将客户端命令作为条目追加到日志，并强制所有 Follower 保持一致
  - 多数派决策：任何决策必须获得超过半数节点同意

Raft 集群的三个角色

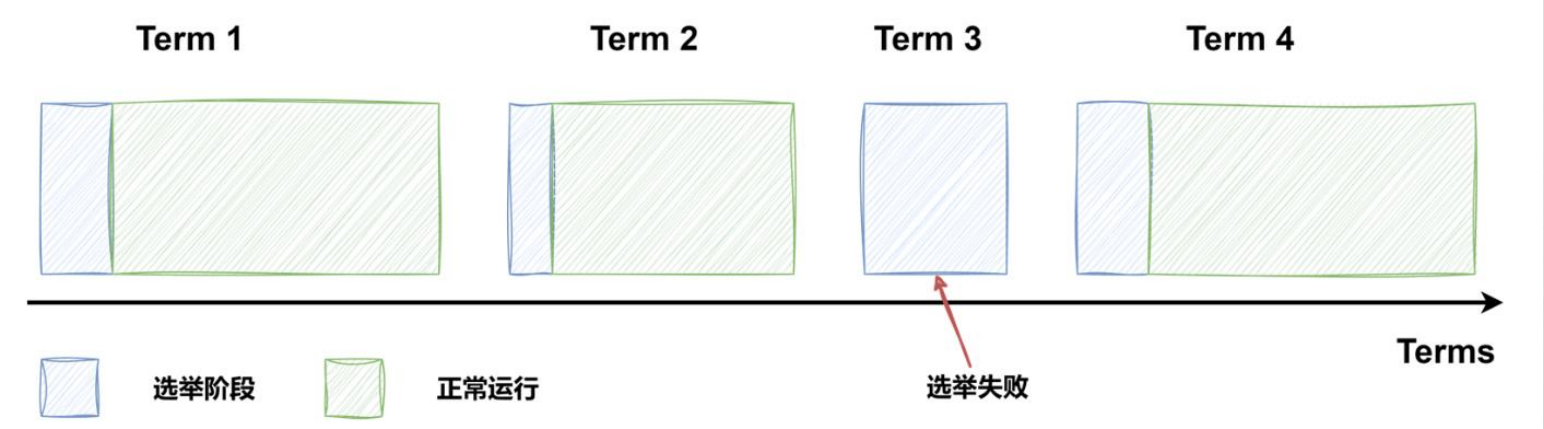
- **三个角色**
  - **Leader**：负责发起心跳，响应客户端，创建日志，同步日志。
  - **Candidate**：Leader 选举过程中的临时角色，由 Follower 转化而来，发起投票参与竞选。
  - **Follower**：被动响应 Leader 和 Candidate 的消息，不发起任何主动操作。
- **服务器的角色转换**



武漢大學
WUHAN UNIVERSITY

任期

- Raft 算法将时间划分为任意长度的任期（term），用连续数字表示。
- 一个任期最多只能有一个 Leader



日志

日志

- 日志包括一系列条目。日志只能追加新条目，不能修改已提交的条目
- 条目内容为<term,index,cmd>。
  - 索引（index）：从 1 开始的连续整数，表示在日志中的位置
  - 任期（term）：该条目被创建时的 Leader 任期
  - 命令（command）：状态机要执行的操作（如 set x=3）
- 所有节点按相同顺序应用日志条目，来保证状态机一致

日志

索引值	1	2	3	4	5	6	7	8	
任期编号 / 指令	1 x <- 3	1 y <- 1	1 y <- 9	2 x <- 2	3 x <- 0	3 y <- 7	3 x <- 5	3 x <- 4	领导者
	1 x <- 3	1 y <- 1	1 y <- 9	2 x <- 2					追随者
	1 x <- 3	1 y <- 1	1 y <- 9	2 x <- 2	3 x <- 0	3 y <- 7			追随者
	1 x <- 3	1 y <- 1							追随者
	1 x <- 3	1 y <- 1	1 y <- 9	2 x <- 2	3 x <- 0	3 y <- 7	3 x <- 5		追随者

武漢大學
WUHAN UNIVERSITY

RPC请求

- **请求投票（RequestVote RPC）**
  - 由 Candidate 在选举期间调用，请求其他节点给自己投票
  - 包含：term、candidateId、lastLogIndex、lastLogTerm
- **添加条目（AppendEntries RPC）**
  - 由 Leader 调用，用于心跳（entries 为空）或日志复制（entries 非空）
  - 包含：term、leaderId、prevLogIndex、prevLogTerm、entries、leaderCommit

Raft 只定义了两种 RPC，比 Paxos 简单很多。心跳和日志复制使用同一个 RPC，只是 entries 字段不同。

拜占庭将军问题

- **问题**
  - 假设多位拜占庭将军需要选举一个大将军，决定是否进攻
  - 假设将军中没有叛军，信使信息可靠但有可能被暗杀的情况下，如何达成选举？
- **解决思路**
  - 假如现在一共有 5 个将军 A、B、C、D、E，每个将军都有一个随机时间的倒计时器
  - A将军倒计时结束，就自荐为大将军候选人，派信使将选举投票请求送给其他将军
  - 其他将军接到请求后，如果没有自荐（自己的倒计时还没有结束），并且没有把选举票投给其他人，它们就会把票投给将军 A
  - 将军 A 自己收到半数以上票数，成为大将军。然后再去派信使通知其他将军。如果一段时间没收到回复（信使可能会被暗杀），就再重派一个信使，直到收到回复。

Raft算法的子问题

- Raft实现了和Paxos相同的功能，它将一致性分解为多个子问题
  - Leader选举(Leader election)
  - 日志复制(Log replication)
  - 安全性(Safety)
  - 日志压缩(Log compaction)
  - 成员变更(Membership change)

动画演示：http://www.kailing.pub/raft/index.html

领导者选举

选举触发

- **心跳**：集群中Leader会向所有Followers周期性发送heartbeat。
- **超时**：Follower如果能收到来自 Leader 或者 Candidate 的有效信息，会保持 Follower 状态，并刷新自己的 electionElapsed（150毫秒至300毫秒的随机值）重新计时。electionElapsed变为0，则表示超时。
- **自荐**：Follower超时后，会等待一段随机时间进行自荐。将自己转为Candidate（term加1）。先给自己投票，并给其他服务器发投票请求

随机超时是 Raft 的精巧设计之一。它保证了几乎不会出现两个 Candidate 同时发起选举，从而避免选票分裂。

领导者选举

选举过程

- **Follower收到 RequestVote 请求后**
  - Term 检查：如果请求的 term < 自己的 currentTerm，拒绝投票
  - 投票者检查：如果自己已经投过票（votedFor != null），且不是投给该 candidate，拒绝
  - 日志比较：candidate 的日志如果比自己旧（比较term和index），拒绝。
- **如果此时Candidate 收到自称Leader的 心跳消息：**
  - 该 Leader 的 term 号大于等于自己的 term 号，则自己回退为 Follower。
  - 该 Leader 的 term 号小于自己的 term 号，拒绝该请求并让该节点更新 term。

日志比较规则是 Raft 安全性的关键：它保证了只有包含所有已提交日志的节点才有可能成为 Leader，从而不会覆盖已提交的日志。

领导者选举

- **选举结果**
  - 赢得了多数（超过1/2）的选票，成功选举为Leader；
  - 收到了Leader的消息，表示有其它服务器已经抢先当选了Leader；
  - 没有Server赢得多数选票，选举失败，等待选举时间超时后发起下一次选举（term++）
- **选举结束**
  - **Candidate** 成为 Leader 后，立即想其他节点发送心跳（AppendEntries）来宣布权威

多数派保证了一个任期内最多只有一个 Leader。如果选票分裂（比如两个 Candidate 各得 1 票），它们都会超时后重试，由于是随机超时，通常很快会选出 Leader。

日志复制

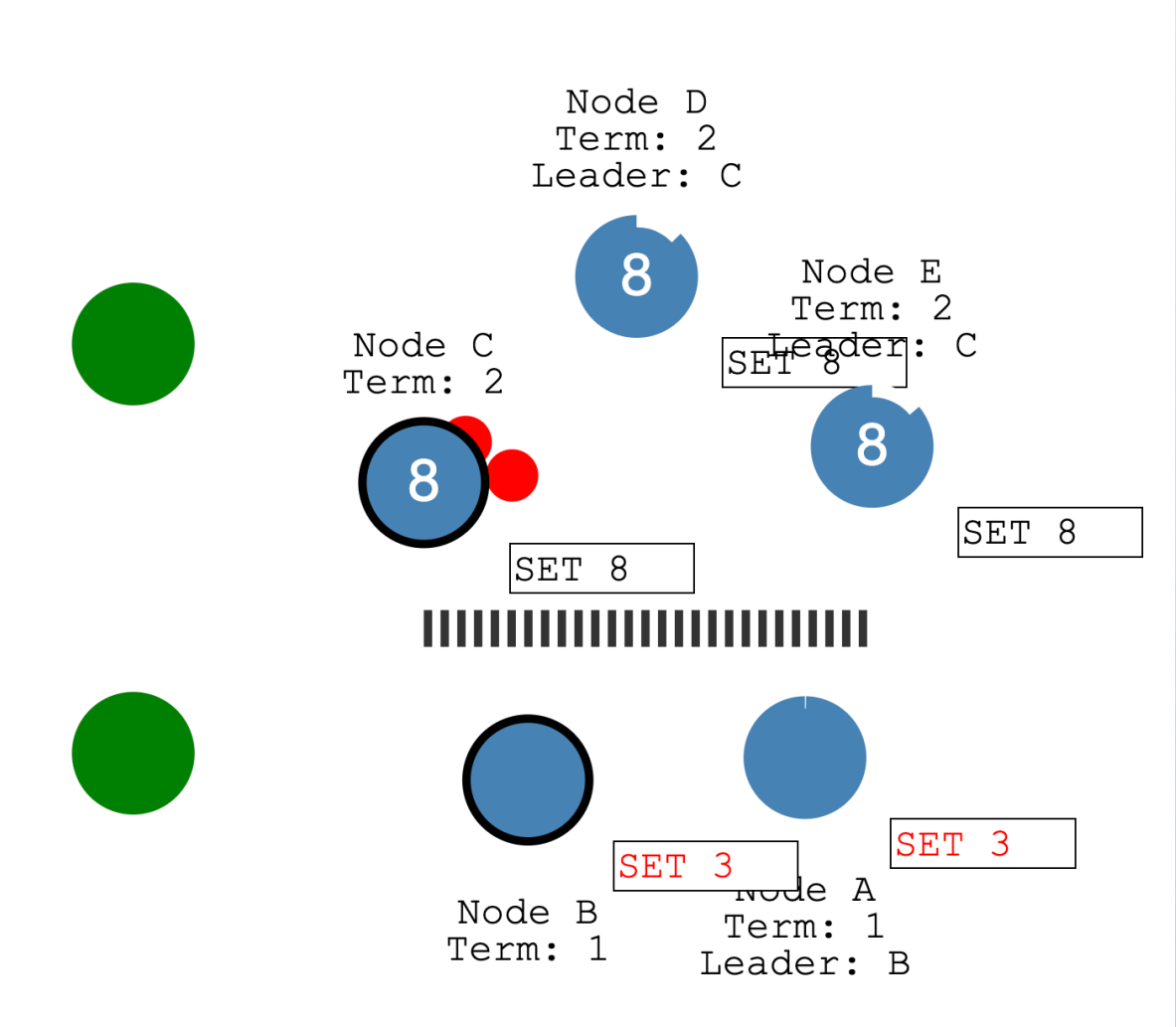
日志复制

- 客户端发送命令到 Leader
- Leader 将命令作为新日志条目追加到本地日志（状态为uncommitted）
- Leader 并行发送 AppendEntries RPC 给所有 Follower
- 当 Leader 确认多数派已复制该条目后，提交（commit）该条目（应用到状态机）
- Leader 在下一个心跳或 AppendEntries 中通知 Follower自己的commitIndex
- Follower 将新提交的日志应用到状态机

- 一个AppendEntries RPC中可以包括多个日志条目，进行批量复制。
- 日志复制过程保证了所有节点最终拥有相同的日志序列。只有 Leader 可以决定提交，Follower 被动接受。

日志复制一致性

- **场景**
  - 发送网络分区时，会出现多位领导者。
  - 这时只有多数节点的分区能够成功完成数据复制
  - 造成两个分区的日志条目的term和index不同



日志复制一致性

- **Raft算法保障了如下性质：**
  - **前缀匹配**：如果两个节点的日志在相同索引处的条目任期相同，那么这两个条目之前的所有日志都相同。
  - **提交连续性**：如果节点某个日志条目状态是已提交，那么所有之前的日志条目状态都是已提交。
- **检测日志不一致**
  - AppendEntries RPC 中包含 prevLogIndex 和 prevLogTerm
  - Follower 收到后，检查prevLogIndex是否存在，索引处的prevLogTerm是否匹配。

日志复制一致性

日志冲突解决

- 若检测不通过，Follower 拒绝该 RPC，Leader 会递减 nextIndex 重试，重复直到找到匹配点。
- 一旦找到匹配点，Follower 会删除冲突条目，追加 Leader 发来的新日志条目

Leader: [1] [2] [3] [4] [5]
Follower: [1] [2] [4] [5] ← 索引3不匹配（4和5是未提交日志）
▲
Leader 发送 AppendEntries(prevLogIndex=3, prevLogTerm=2)
Follower 检查发现索引3的term不匹配（Leader是2，Follower是4），拒绝
Leader 递减 nextIndex 到 3，发送 prevLogIndex=2, prevLogTerm=2
Follower 匹配，接受后续条目 [3][4][5]，覆盖自己冲突的部分

Raft 采用 ‘Leader 强制覆盖’ 的策略。虽然这可能导致未提交的日志被覆盖，但这是安全的，因为只有已提交的日志才不能被覆盖。因为未提交的日志本来就没有被多数认可

状态机安全性

- **定理**
  - 如果某个节点已将某日志条目应用到状态机，其他节点不会在相同索引处应用不同命令。
- 保证条件
  - 选举限制（只选日志最新的节点为 Leader）
  - 日志匹配性质（Leader 强制复制）
  - 提交规则（Leader 只提交自己任期内的日志）
  - 结果：Raft 实现了线性一致性（Linearizability）

这些规则共同保证了所有节点按相同顺序执行相同的命令，从而得到相同的状态。这是分布式系统正确性的基础。

成员变更

挑战

- 问题：直接切换集群配置（如增加节点，或则减少节点）可能导致两个多数派同时决策
- 场景：旧配置 C_old 和新配置 C_new 同时存在，可能选出两个 Leader
- Raft 解决方案：联合共识（Joint Consensus）

成员变更必须谨慎。如果直接切换配置，可能会产生两个互不重叠的多数派，导致两个 Leader 同时存在。Raft 使用两阶段方法解决。

成员变更

联合共识

- 第一阶段：
  - Leader 发起 C_old,new 配置（联合配置）。包含新旧所有节点
  - 任何决策需要同时获得 Cold 和 Cnew 两个配置各自的多数派同意
- 第二阶段：
  - 联合配置生效后再发起 C_new 配置
  - 只需要新配置的多数派
- 安全保证：在转换期间，不会产生两个不相交的多数派

日志压缩——快照

- **问题**
  - 日志无限增长会消耗大量存储，重启时重放日志时间过长
- **快照**
  - 将状态机**当前状态写入持久存储**，丢弃之前的所有日志
  - 每个节点可以独立制作快照，Leader 也可以发送快照给慢速 Follower
  - 快照内容：状态机状态（如数据库的全部数据）、最后包含的日志索引和任期

快照是工程实践上必须的优化。Raft 的设计允许每个节点独立做快照，而不需要 Leader 协调，这简化了实现。

如何从Raft集群中读数据？

方案

- Raft Log Read：把读请求也当作一条普通的Raft日志来处理
- ReadIndex Read：通过轻量级的"心跳确认"确保Leader身份合法，从而保证线性一致性
- Lease Read/Local Read：Leader与多数节点通信后，获得一个租约（Lease），在租约时间内，Leader的身份是绝对有效的。

方案	强一致性	网络I/O	磁盘I/O	典型场景
Raft Log Read	✓ 强	高（日志复制）	高（日志落盘）	几乎不用，作为理论基准
ReadIndex Read	✓ 强	中（Leader确认）	无	延迟敏感，且无法保证时钟稳定的通用场景
Lease Read	✓ 强	无（租约期内）	无	高并发、低延迟，且时钟可靠的性能敏感场景

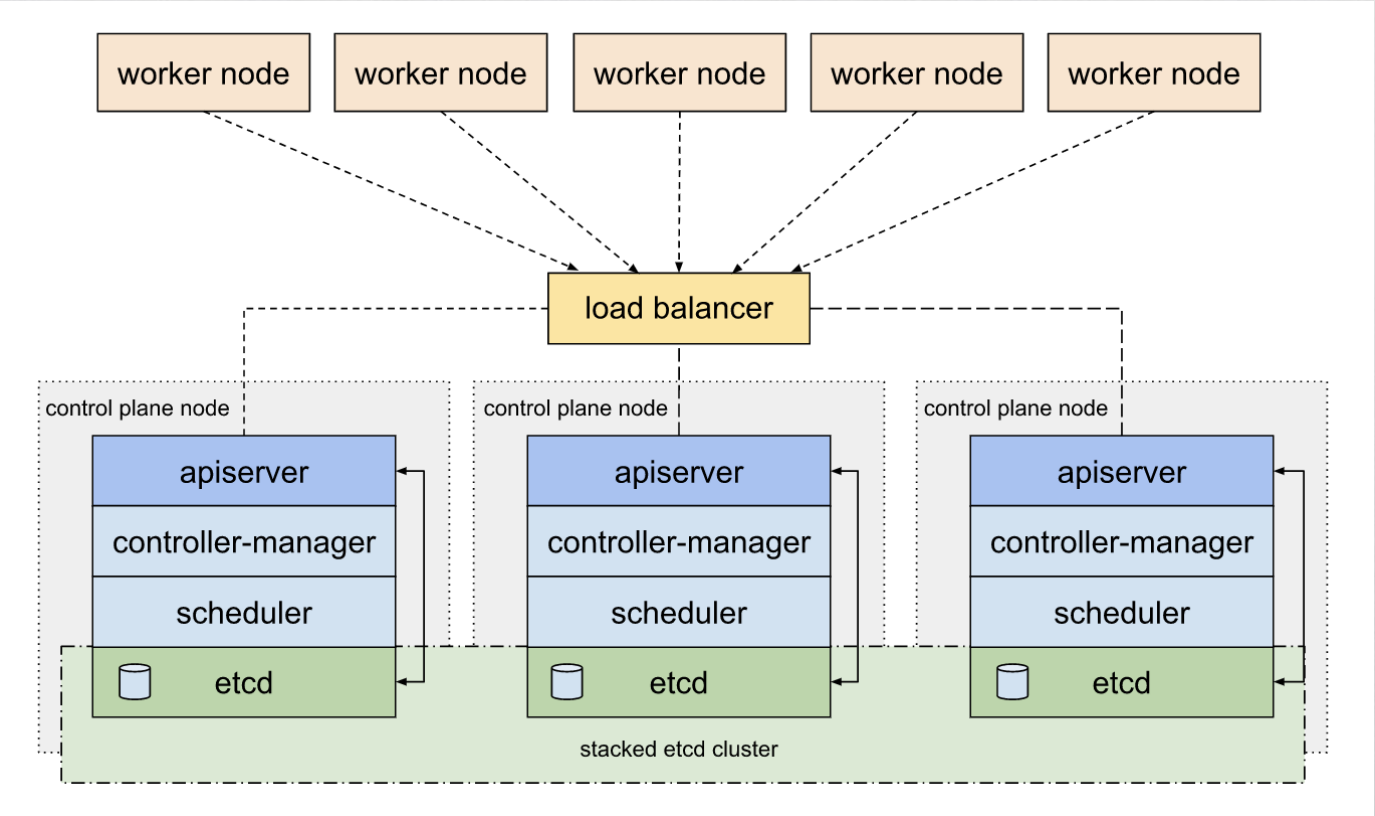
Paxos和Raft的比较

维度	Paxos	Raft
设计目标	理论正确性	可理解性 + 工程实践
角色	Proposer/Acceptor/Learner	Leader/Follower/Candidate
领导者	可选（Multi-Paxos）	必须有且只有一个
日志组织	提案编号	连续日志索引 + Term
选举机制	无内置，需额外设计	随机超时 + Term 比较
成员变更	复杂，需两阶段	联合共识 + 配置变更
工程生态	少，实现难度大	丰富（etcd, Consul, TiKV）

Raft算法的工业应用

- Etcd：Kubernetes 的核心存储，使用 Raft 保证元数据一致性
- Consul：服务发现与配置管理
- TiKV：分布式 KV 数据库，底层使用 Raft
- CockroachDB：分布式 SQL 数据库
- MongoDB：部分复制集功能基于 Raft 思想
- RabbitMQ：队列元数据使用 Raft

Etcd如何管理Kubernetes元数据？

Etcd的作用

- 存储了Kubernetes集群所有核心资源状态信息，包括节点信息、Pod与服务、配置与密钥、访问控制、集群元数据等
- Etcd内部使用 Raft共识算法来保证数据在多个节点间强一致。这意味着集群中任一节点看到的数据都完全相同，避免了“脑裂”和数据冲突。